ETHEREUM
ECONOMIC
ZONE

ZisK VM

Workshop
EEZ Architecture

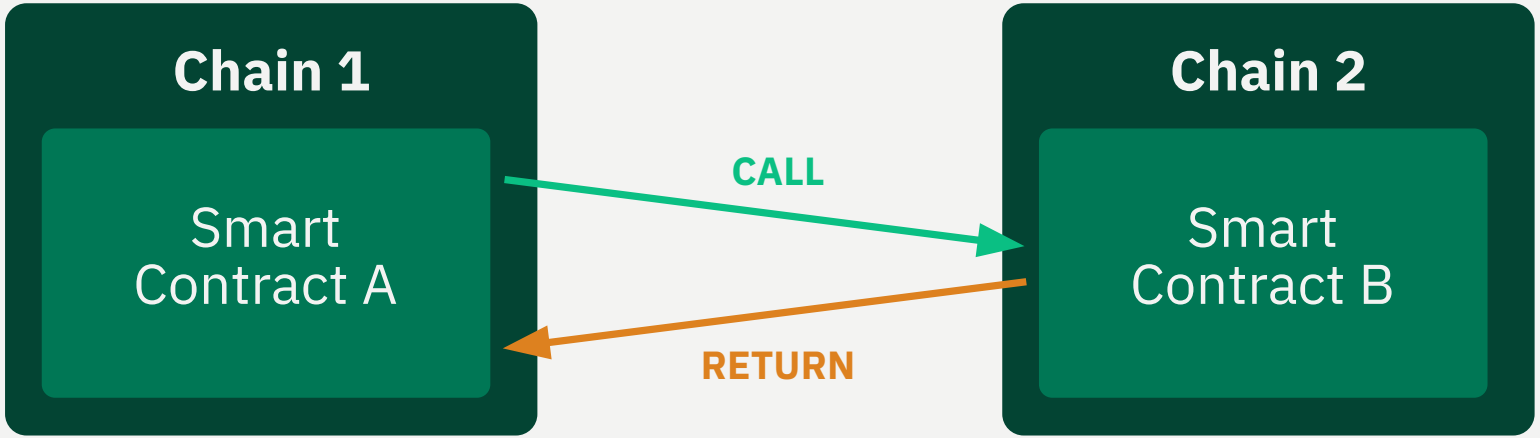
DAPPCon EEZ Workshop, 17 June 2026

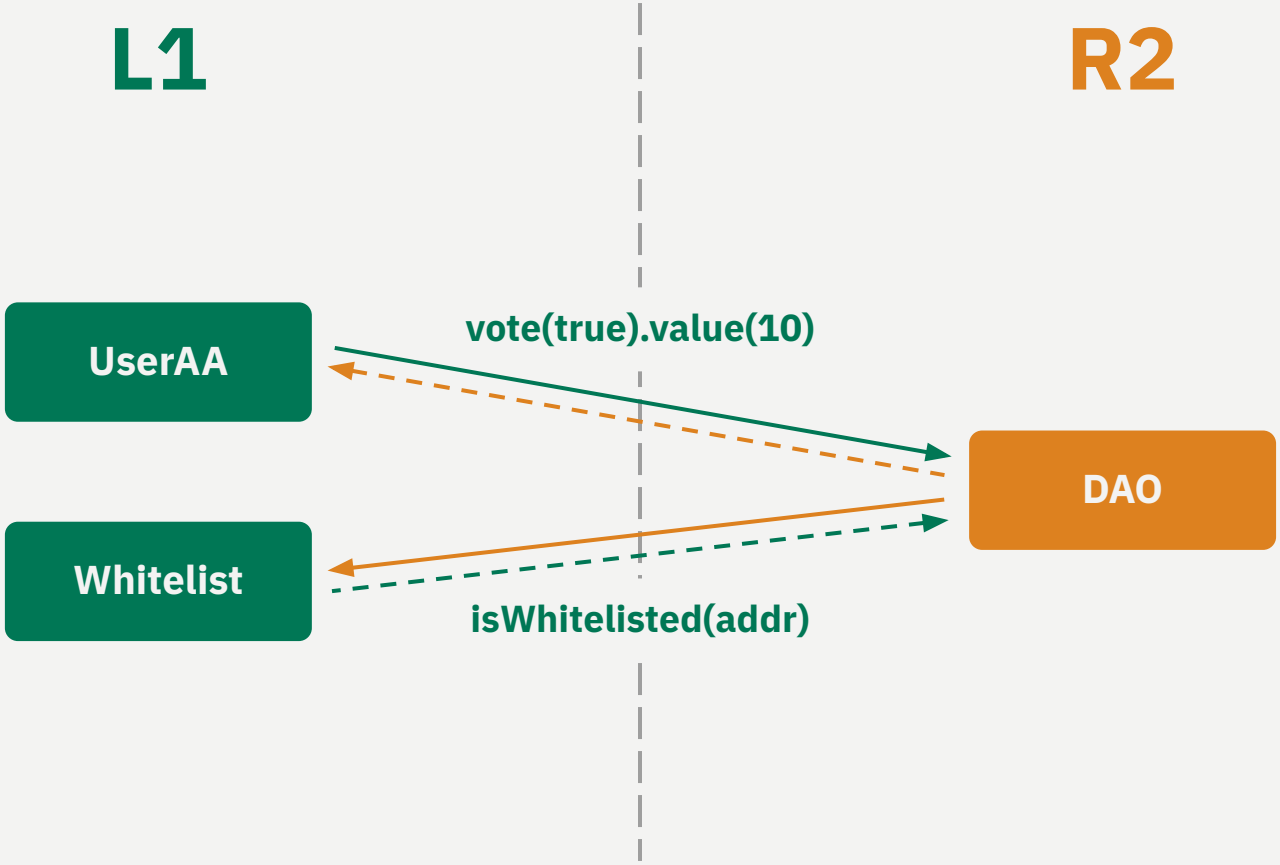
@jbaylina

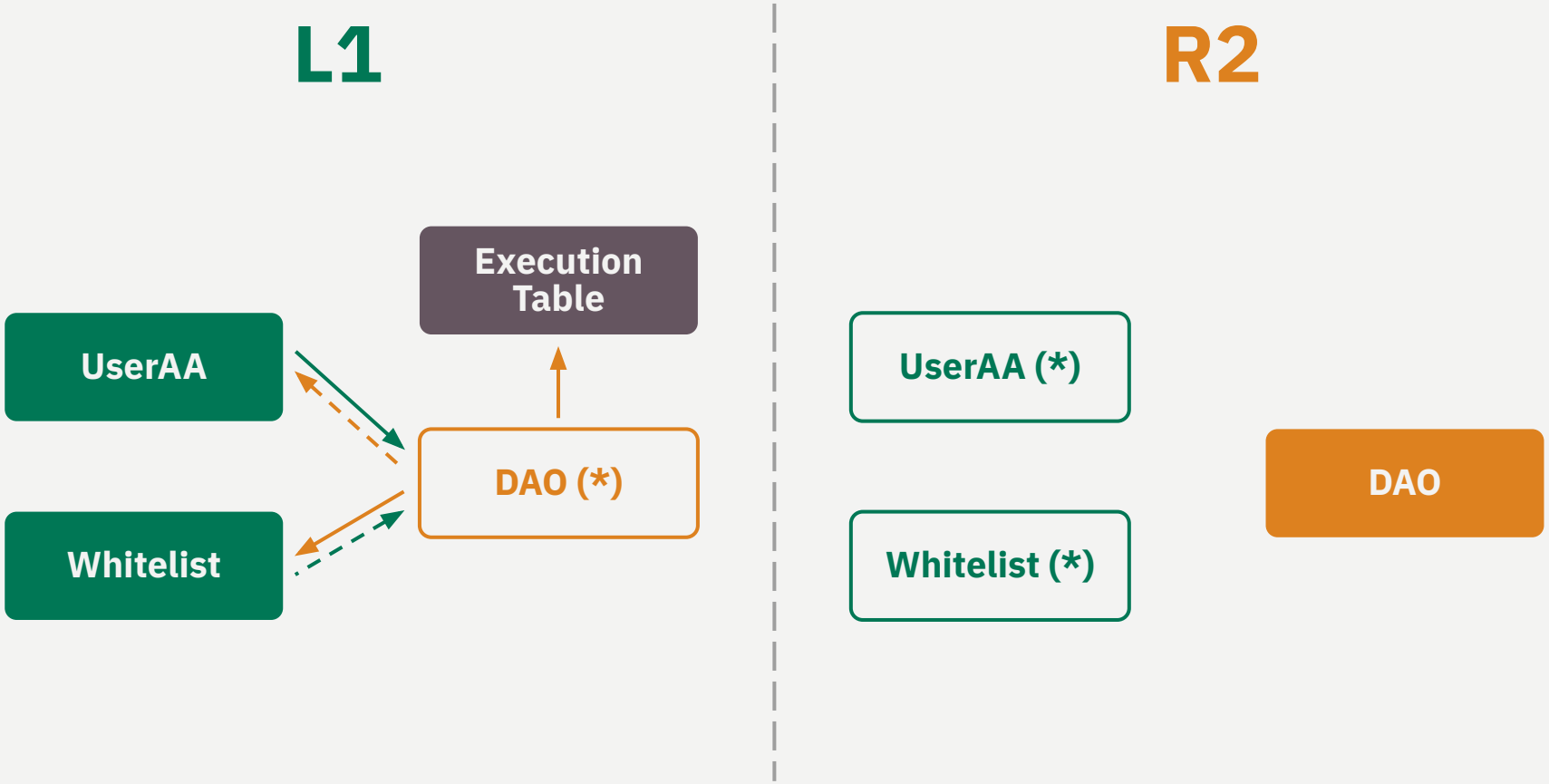
THE EEZ PROOF

What is EEZ?

EEZ proves the combined execution of many rollups as a single, synchronous transaction.




L1
R2
UserAA
vote(true).value(10)
DAO
Whitelist
isWhitelisted(addr)


L1
R2
Execution Table
UserAA
DAO (*)
Whitelist
UserAA (*)
Whitelist (*)
DAO

EEZ Properties

01 **Permissionless smart contract**

02 **No upgradable**

03 **Roll-ups can opt-in, opt-out freely**

04 **Proof-system agnostic**

- ZK based
- Tee
- Multisig

05 **Rollups are sovereign by themselves, governed by a rollup smart contract**

- Each rollup defines its rules in a smart contract.
- Each rollup defines its own state transformation function
- Each rollup define the accepted proving systems.

06 **Security warranted across rollups**

- Same security that Ethereum provides to independent smart contracts calling one each other.

07 **Orthogonal to L1 pre-confirmations**

Custom native rollups

01

Each rollup can have its own state transition function (WASM, RISCV, custom defined STF...)

02

The interaction with other rollups is executed via normal Ethereum CALLs

03

If rollups have ETH as is native token, CALLs can include ETH.

04

For ETH transfers between rollups, a rollup-level accounting is maintained in L1.

05

Rollups are sovereign. They can define their update mechanism.

06

Permissionless rollup creation.

Based / Centralized Sequencer Rollups

- Any centralized rollup can be treated as a base rollup with all its state transitions signed by the centralized sequencer.

- It’s the responsibility of the L2 sequencer to coordinate with the composer to include txs that touches other chain. Some sort of locking mechanism or reorgs is required.
  - Optimistic method: reorgs.
  - Pessimistic method: locking (not necessarily the full chain)

Validiums

- Validiums can decide to not include self-chain only TXs in the blobs. That’s fine, but they should provide that info or some coordination mechanisms with composers if they want their TXs to touch other chains.

The composer

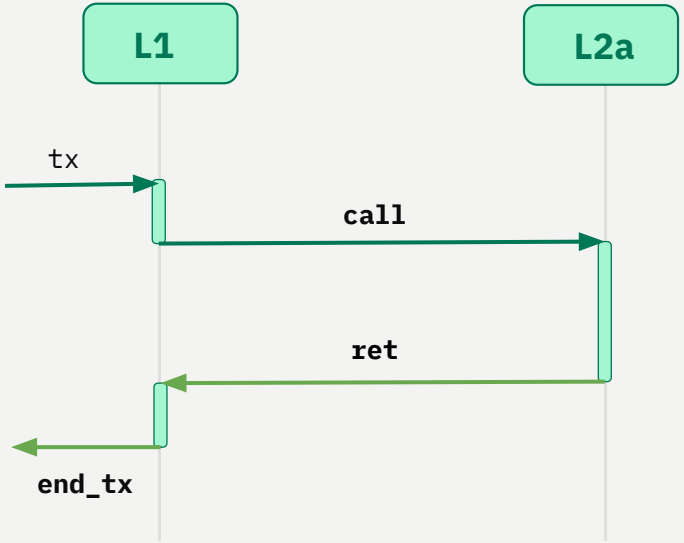
- Builds Sends the postAndVerifyBatch transaction/bundle.
- Any body can be a composer.
- EEZ does not define currently a specific incentive mechanism for sending fees to the composer. But we expect TX will bring implicit/explicit reward to the composer so it has the incentives to include the TXs.
- EEZ does not define currently a specific public pool of crosschain TXs

Clients

- A client that’s following a single L2 chain should be able to build it’s state without synchronizing any other L2 (Only L1 and his chain)

EEZ Trace; The blob format

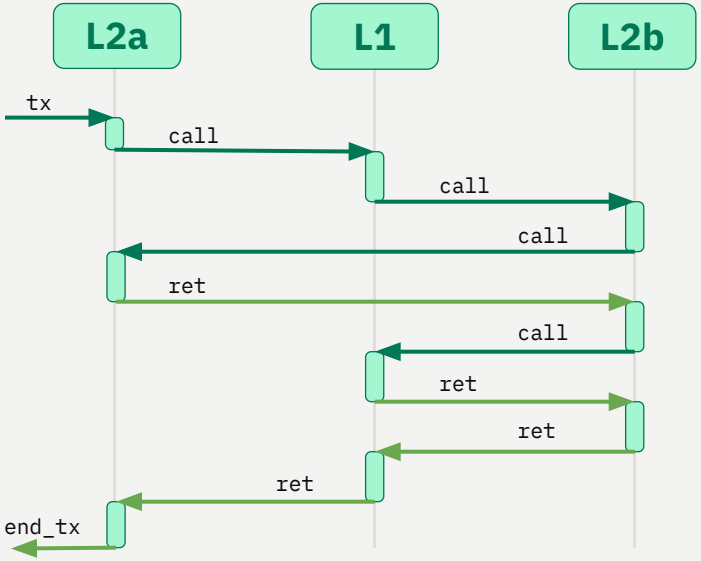
It records each context switch between chains (call and returns).



type	from	to	info
tx	L1		rawTX
call	L1	L2a	from, to, value, data
return	L2a	L1	success, retData
end_tx	L1		success

EEZ Trace: Across Rollups

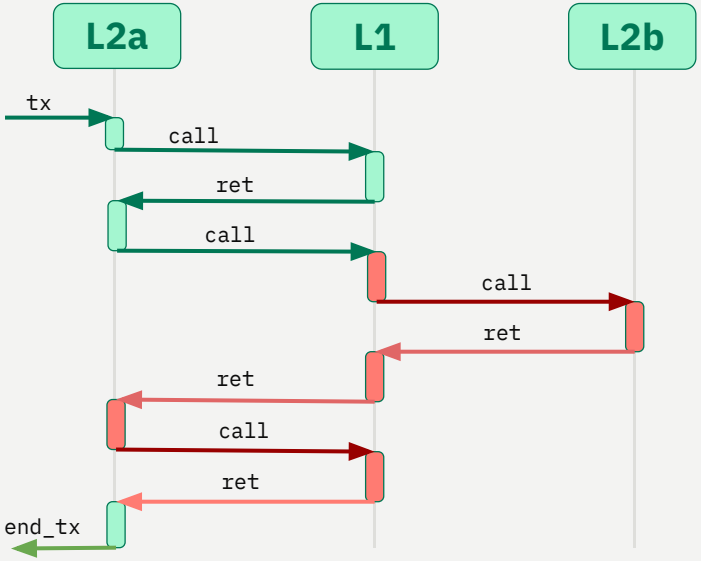
It records each context switch between chains (call and returns).



type	from	to	info
tx		L2a	rawTX
call	L2a	L1	from, to, value, data
call	L1	L2b	from, to, value, data
call	L2b	L2a	from, to, value, data
ret	L2a	L2b	success, retData
call	L2b	L1	from, to, value, data
ret	L1	L2b	success, retData
ret	L2b	L1	success, retData
ret	L1	L2a	success, retData
end_tx	L2a	–	success

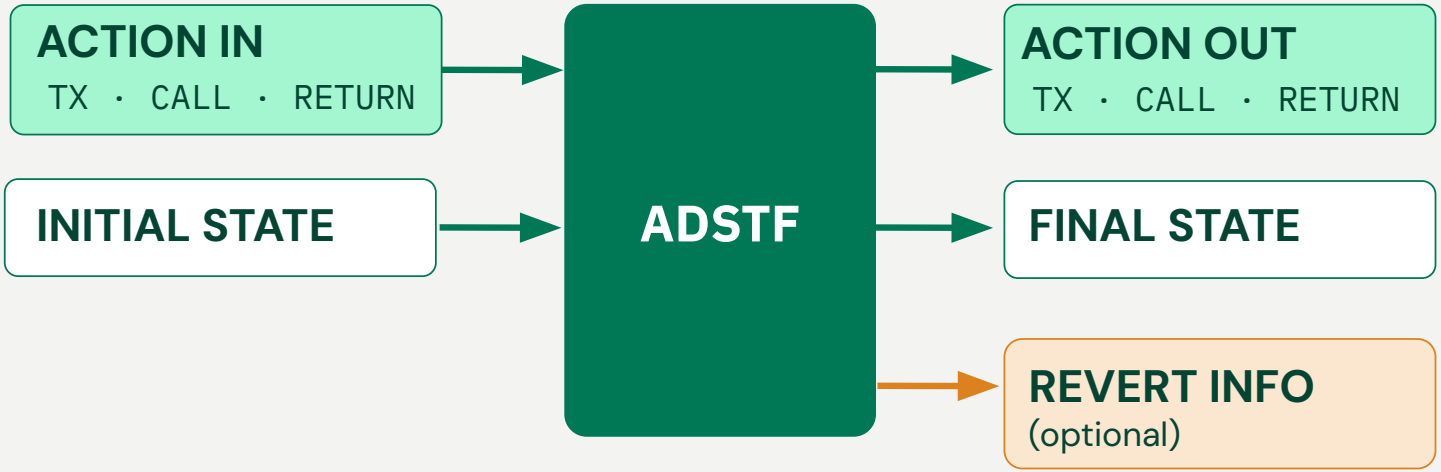
EEZ Trace: Across Rollups (revert)

It records each context switch between chains (call and returns).



type	from	to	info
tx		L2a	rawTX
call	L2a	L1	from, to, value, data
return	L1	L2a	success, retData
call	L2a	L1	from, to, value, data
call	L1	L2b	success, retData
return	L2b	L1	success, retData
return	L1	L2a	success, retData
call	L2a	L1	success, retData
return	L1	L2a	success, retData
revert	L2a	L2a	called(-2)
end_tx	L2a	-	success

Action-Driven State Transition Function



```
function registerRollup(
    address rollupContract,
    bytes32 initialState)
external returns (uint256 rollupId) {
    if (rollupContract == address(0) || rollupContract == address(this))
      revert InvalidRollupContract();

    rollupId = ++rollupCounter;
    rollups[rollupId] = RollupConfig({
        rollupContract: rollupContract,
        stateRoot: initialState,
        etherBalance: 0
    });

    IRollupContract(rollupContract).rollupContractRegistered(rollupId);

    emit RollupCreated(rollupId, rollupContract, initialState);
}
```

```
interface IRollupContract {
   function rollupContractRegistered(uint256 rollupId)
      external;
   function checkProofSystemsAndGetVkeys(address[] calldata proofSystems)
      external view returns (bytes32[] memory vkeys);
   function getCustomData(uint64 blockNumber)
      external view returns (bytes customData);
}
```

```
struct ProofSystemBatchPerVerificationEntries{
    …
    address[] proofSystems;
    RollupIdWithProofSystems[] rollupIdsWithProofSystems
    …
```

```
struct RollupIdWithProofSystems {
    uint256 rollupId;
    uint64[] proofSystemIndex;
}
```

```
    function _fetchVkMatrix(ProofSystemBatchPerVerificationEntriescalldata batch)
        internal
        view
        returns (bytes32[][] memory vkMatrix)
    {
        vkMatrix = new bytes32[][](batch.rollupIdsWithProofSystems.length);
        for (uint256 r = 0; r < batch.rollupIdsWithProofSystems.length; r++) {
            RollupIdWithProofSystemscalldata rps = batch.rollupIdsWithProofSystems[r];
            uint64[] calldata indices = rps.proofSystemIndex;

            address[] memory proofSystemUsed = new address[](indices.length);
            for (uint256 j = 0; j < indices.length; j++) {
                proofSystemUsed[j]= batch.proofSystems[uint256(indices[j])];
            }

            vkMatrix[r] =
                IRollupContract(rollups[rps.rollupId].rollupContract)
                    .checkProofSystemsAndGetVkeys(proofSystemUsed);
            if (vkMatrix[r].length != indices.length) revert InvalidProofSystemConfig();
        }
    }
```

```
function postAndVerifyBatch(ProofSystemBatchPerVerificationEntries calldata batch) external
```

```
struct ProofSystemBatchPerVerificationEntries {
   ExecutionEntry[] entries;
   LookupCall[] l1ToL2lookupCalls;
   uint256 transientExecutionEntryCount;
   uint256 transientLookupCallCount;
   address[] proofSystems;
   RollupIdWithProofSystems[] rollupIdsWithProofSystems;
   uint256[] blobIndices;
   bytes callData;
   bytes[] proofs;
   uint64 blockNumber;
}
```

```
interface IProofSystem {
   function verify(bytes calldata proof, bytes32 publicInputsHash) external view returns (bool
valid);
}
```

```
function _verifyProofSystemBatch(ProofSystemBatchPerVerificationEntriescalldata batch, bytes32[][]
      memory vkMatrix) internal view {
       // Selected blob hashes (indexed into the tx-level blob set)
       bytes32[] memory blobHashes = new bytes32[](batch.blobIndices.length);
       for (uint256 i = 0; i < batch.blobIndices.length; ++i) {
           blobHashes[i] = blobhash(batch.blobIndices[i]);
       }

       // Per-entry hash binds the FULL entry content: stateDeltas, proxyEntryHash,
       // destinationRollupId, l2ToL1Calls[], expectedL1ToL2Calls[], expectedLookups[],
       // callCount, returnData, rollingHash. Prevents an orchestrator from swapping
       // call/reentrant-call/returnData at execution time without invalidating the proof.
       bytes32[] memory entryHashes = new bytes32[](batch.entries.length);
       for (uint256 i = 0; i < batch.entries.length; ++i) {
           entryHashes[i] = keccak256(abi.encode(batch.entries[i]));
       }

       // Per-lookup-call hash, same rationale.
       bytes32[] memory lookupCallHashes = new bytes32[](batch.l1ToL2lookupCalls.length);
       for (uint256 i = 0; i < batch.l1ToL2lookupCalls.length; ++i) {
           lookupCallHashes[i] = keccak256(abi.encode(batch.l1ToL2lookupCalls[i]));
       }

       bytes32 sharedPublicInput = keccak256(
           abi.encodePacked(
               abi.encode(entryHashes),
               abi.encode(lookupCallHashes),
               abi.encode(blobHashes),
               keccak256(batch.callData),
           )
       );
```

```
        // Fetch (timestamp, blockHash) for every rollup ONCE — reused across all PS loops
        // below. Both reads are `view` (STATICCALL) so a malicious manager cannot reenter.
        uint256 rollupCount = batch.rollupIdsWithProofSystems.length;
        bytes[] memory extraData = new bytes[](rollupCount);
        for (uint256 r = 0; r < rollupCount; r++) {
            extraData[r] = IRollupContract(
                    rollups[batch.rollupIdsWithProofSystems[r].rollupId].rollupContract
                ).getExtraData(batch.blockNumber);
        }

        // Per-PS verification — for each PS k, walk attesting rollups in canonical order
        // (rollupId-ascending, the order the batch enforces) and fold each rollup's
        // (rollupId, vkey for PS k, blockHash, timestamp) into a rolling accumulator. Off-
        // chain provers MUST mirror this incremental scheme so the on-chain rebuild matches.
        for (uint256 k = 0; k < batch.proofSystems.length; k++) {
            bytes32 acc = bytes32(0);
            for (uint256 r = 0; r < rollupCount; r++) {
                RollupIdWithProofSystemscalldata rps = batch.rollupIdsWithProofSystems[r];
                uint256 j =  findIndexPosition(rps.proofSystemIndex, k);
                if (j == type(uint256).max) continue;
                acc = keccak256(abi.encode(acc, rps.rollupId, vkMatrix[r][j], extraData[r]));
            }

            bytes32 publicInputsHash = keccak256(abi.encodePacked(sharedPublicInput, acc));

            if (!IProofSystem(batch.proofSystems[k]).verify(batch.proofs[k], publicInputsHash)) {
                revert InvalidProof();
            }
        }
    }
```

```
struct ExecutionEntry {
    StateDelta[] stateDeltas;
    bytes32 proxyEntryHash;
    uint256 destinationRollupId;
    L2ToL1Call[] l2ToL1Calls;
    ExpectedL1ToL2Call[] expectedL1ToL2Calls;
    uint256 callCount;
    bytes returnData;
    bytes32 rollingHash;
}
```

Some examples

Examples:

- L2 only transactions
- L1 call L2
- L2 call L1
- L1 static_call L2
- L2 static_call L1
- L1 call L2 call L1
- L2 call L1 call L2
- L1 call L2 callstatic L1
- L2 call L1 callstatic L2
- L1 callstatic L2 callstatic L1
- L2 callstatic L1 callstatic L2
- L2a call L1 call/revert L2b
- L2a call L1, L2a call L1 call L2b, L2a call L1 revert 2 last
- Nested L1 call L2a call L1 call L2b call L1

Single L2 only tx

```
proofSystemBatchPerVerificationEntries = {
   entries = [{
      stateDeltas = [{
         rollupId: 1,
         currentState: 0x1234,
         nextState: 0x5678,
         etherDelta: 0
      }]
      proxyEntryHash = 0,
      destinationRollupId = 1,
      returnData = {},
      l2ToL1Calls = [],
      expectedL1ToL2Calls = [],
      expectedLookups = [],
      callCount = 0,
      rollingHash = 0,
   }],
   l1ToL2lookupCalls = [],
   …
}
```

Many L2 transactions

```
proofSystemBatchPerVerificationEntries = {
   entries = [{
      stateDeltas = [{
         rollupId: 1,
         currentState: 0x1234,
         nextState: 0x5678,
         etherDelta: 2
      },{
         rollupId: 2,
         currentState: 0xabcd,
         nextState: 0xef12,
         etherDelta: -2
      }],
      proxyEntryHash = 0,
      destinationRollupId = 0,
      l2ToL1Calls = [],
      expectedL1ToL2Calls = [],
      expectedLookups = [],
      callCount = 0,
      returnData = "",
      rollingHash = 0,
   }],
   l1ToL2lookupCalls = [],
   …
}
```

L1 call L2

```
proofSystemBatchPerVerificationEntries = {
   entries = [{
         stateDeltas = [{
            rollupId: 1,
            currentState: 0x1234,
            nextState: 0x5678,
            etherDelta: 2
         }]
         proxyEntryHash = keccak(
            dest_rollupId: 1,
            targetAddress: 0xADDR_IN_L2,
            value: 2,
            callData: "Data of the L1->L2 call",
            sourceAddress: 0xADDR_IN_L1,
            src_rollupId: MAINNET_ROLLUP_ID=0),
         destinationRollupId = 1,
         returnData = "myReturnData"
         l2ToL1Calls = [],
         expectedL1ToL2Calls = [],
         expectedLookups = [],
         callCount = 0,
         rollingHash = 0,
      }],
   l1ToL2lookupCalls = [],
   …
}
```

L2 call L1

```
proofSystemBatchPerVerificationEntries = {
   entries = [{
         stateDeltas = [{
            rollupId: 1,
            currentState: 0x1234,
            nextState: 0x5678,
            etherDelta: -2
         }]
         proxyEntryHash = 0,
         destinationRollupId = 1,
         returnData = {}
         l2ToL1Calls = [{
            isStatic: false,
            targetAddress: 0x1212,
            value: 2,
            data: 0xaaaa,
            sourceAddress: 0x34343,
            sourceRollupId: 1,
            reverSpan: 0
         }],
         expectedL1ToL2Calls = [],
         expectedLookups = [],
         callCount = 1,
         rollingHash = 0x646464646,
      }],
   l1ToL2lookupCalls = [],
   …
}
```

```
struct L2ToL1Call {
   bool isStatic;
   address targetAddress;
   uint256 value;
   bytes data;
   address sourceAddress;
   uint256 sourceRollupId;
   uint256 revertSpan;
}
```

L1 static call L2

```
proofSystemBatchPerVerificationEntries = {
   entries = [],
   l1ToL2lookupCalls = [{
      crossChainCallHash = keccak(
            dest_rollupId: 1,
            originalAddress: 0x1234,
            value: 0,
            callData: 0xaaaaaa,
            sourceAddress: 0x5678
            src_rollupId: MAINNET_ROLLUP_ID=0),
      destinationRollupId = 1,
      returnData = "myStaticReturn",
      failed = false,
      l2ToL1Calls =[],
      expectedL1ToL2Calls = [],
      expectedLookups = [],
      callCount = 0,
      rollingHash = 0,
      expectedStateRoots = [{
            rollupId: 1
            stateRoot: 0x454545
      ]}
   }],
   …
}
```

```
struct LookupCall {
   bytes32 crossChainCallHash;
   uint256 destinationRollupId;
   bytes returnData;
   bool failed;
   L2ToL1Call[] l2ToL1Calls;
   ExpectedL1ToL2Call[] expectedL1ToL2Calls;
   ExpectedLookup[] expectedLookups;
   uint256 callCount;
   bytes32 rollingHash;
   ExpectedStateRootPerRollup[] expectedStateRoots;
}
```

L2 static call L1

```
proofSystemBatchPerVerificationEntries = {
   entries = [{
         stateDeltas = [{
            rollupId: 1,
            currentState: 0x1234,
            nextState: 0x5678,
            etherDelta: 0
         }]
         proxyEntryHash = 0,
         destinationRollupId = 0,
         returnData = {}
         l2ToL1Calls = [{
            isStatic: true,
            targetAddress: 0xL2_ADDR,
            value: 0,
            data: 0xaaaa,
            sourceAddress: 0xL1_ADDR,
            sourceRollupId: 1,
            reverSpan: 0
         }],
         expectedL1ToL2Calls = [],
         expectedLookups = [],
         callCount = 1,
         rollingHash = 0x646464646,
      }],
   l1ToL2lookupCalls = [],
   …
}
```

L1 call L2 call L1

```
proofSystemBatchPerVerificationEntries = {
   entries = [{
          stateDeltas = [{
              rollupId: 1,
              currentState: 0x1234,
              nextState: 0x5678,
              etherDelta: 0
          }]
          proxyEntryHash = keccak(
              dest_rollupId: 1,
              originalAddress: 0x11111a,
              value: 2,
              callData: 0x22222,
              sourceAddress: 0x111111b,
              src_rollupId: MAINNET_ROLLUP_ID=0),
          destinationRollupId = 1,
          returnData = "myReturnData"
          l2ToL1Calls = [{
              isStatic: false,
              targetAddress: 0x11111c,
              value: 2,
              data: 0xaaaa,
              sourceAddress: 0x22222,
              sourceRollupId: 1,
              reverSpan: 0
          }],
          expectedL1ToL2Calls = [],
          expectedLookups = [],
          callCount = 1,      // Top Level l2toL1 calls executed.
          rollingHash = 0x646464646,  // Needed for return value
       }],
   l1ToL2lookupCalls = [],
   …
}
```

L2 call L1 call L2

```
proofSystemBatchPerVerificationEntries = {
   entries = [{
          stateDeltas: [{1, 0x1234, 0x5678, 0}],
          proxyEntryHash: 0,
          destinationRollupId: 0,
          returnData: {}
          l2ToL1Calls: [{
              isStatic: false,
              targetAddress: 0x1111,
              value: 2,
              data: 0xaaaa,
              sourceAddress: 0x222a,
              sourceRollupId: 1,
              reverSpan: 0
          }],
          expectedL1ToL2Calls: [
              crossChainCallHash: keccak(
                    dest_rollupId: 1,
                    targetAddress: 0x22222,
                    value: 2,
                    callData: 0xaaaaaa,
                    sourceAddress: 0x1111,
                    src_rollupId: MAINNET_ROLLUP_ID=0),
              callCount: 0
              returnData: "MySecondCallReturn"
          }],
          expectedLookups = [],
          callCount = 1,
          rollingHash = 0x646464646,
       }],
   l1ToL2lookupCalls = [],
   …
}
```

```
struct ExpectedL1ToL2Call {
   bytes32 crossChainCallHash;
   uint256 callCount;
   bytes returnData;
}
```

L1 call L2 call_static L1

```
proofSystemBatchPerVerificationEntries = {
   entries = [{
           stateDeltas = [{
                rollupId: 1,
                currentState: 0x1234,
                nextState: 0x5678,
                etherDelta: 0
           }]
           proxyEntryHash = keccak(
                dest rollupId: 1,
                originalAddress: 0x11111a,
                value: 2,
                callData: 0x22222,
                sourceAddress: 0x111111b,
                src rollupId: MAINNET_ROLLUP_ID=0),
           destinationRollupId = 1,
           returnData = "myReturnData"
           l2ToL1Calls = [{
                isStatic: true,
                targetAddress: 0x11111c,
                value: 2,
                data: 0xaaaa,
                sourceAddress: 0x22222,
                sourceRollupId: 1,
                reverSpan: 0
           }],
           expectedL1ToL2Calls = [],
           expectedLookups = [],
           callCount = 1,
           rollingHash = 0x646464646,
       }],
   l1ToL2lookupCalls = [],
   …
}
```

L2 call L1 static_call L2

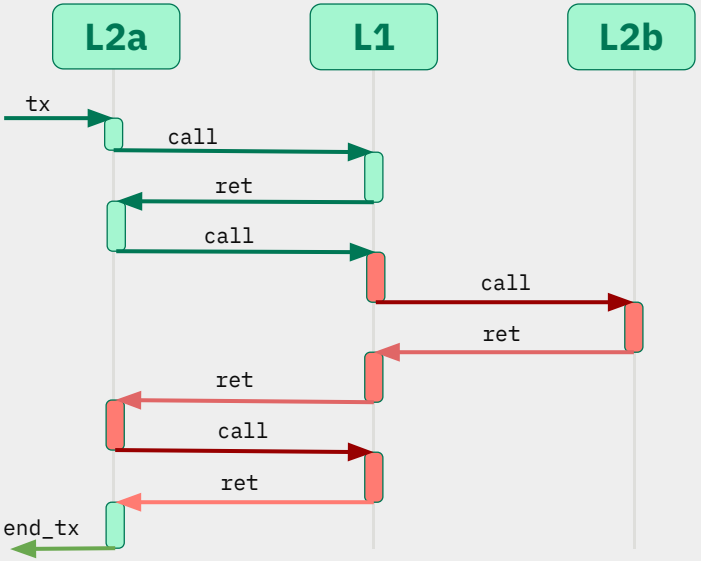
```
proofSystemBatchPerVerificationEntries = {
   entries = [{
          stateDeltas: [{1, 0x1234, 0x5678, 0}],
          proxyEntryHash: 0,
          destinationRollupId: 0,
          returnData: {}
          l2ToL1Calls: [{
               isStatic: false,
               targetAddress: 0x1111,
               value: 2,
               data: 0xaaaa,
               sourceAddress: 0x2222a,
               sourceRollupId: 1,
               reverSpan: 0
          }],
          expectedL1ToL2Calls: [],
          expectedLookups = [{
               crossChainCallHash: keccak(
                      dest_rollupId: 1,
                      targetAddress: 0x2222a,
                      value: 2,
                      callData: 0xaaaaaa,
                      sourceAddress: 0x1111,
                      src_rollupId: MAINNET_ROLLUP_ID=0),
               returnData: "MySecondCallReturnData"
               failed: false
               l2ToL1CallNumber: 0
               lastL1ToL2CallConsumed: 0
               executingLookupIndex: 0
               callCount: 0
               rollingHash: 0
          }],
          callCount = 1,
          rollingHash =  0x646464646,
       }],
   l1ToL2lookupCalls = [],
   …
}
```

```
struct ExpectedLookup {
   bytes32 crossChainCallHash;
   bytes returnData;
   bool failed;
   uint64 l2ToL1CallNumber;
   uint64 lastL1ToL2CallConsumed;
   uint64 executingLookupIndex;
   uint256 callCount;
   bytes32 rollingHash;
}
```

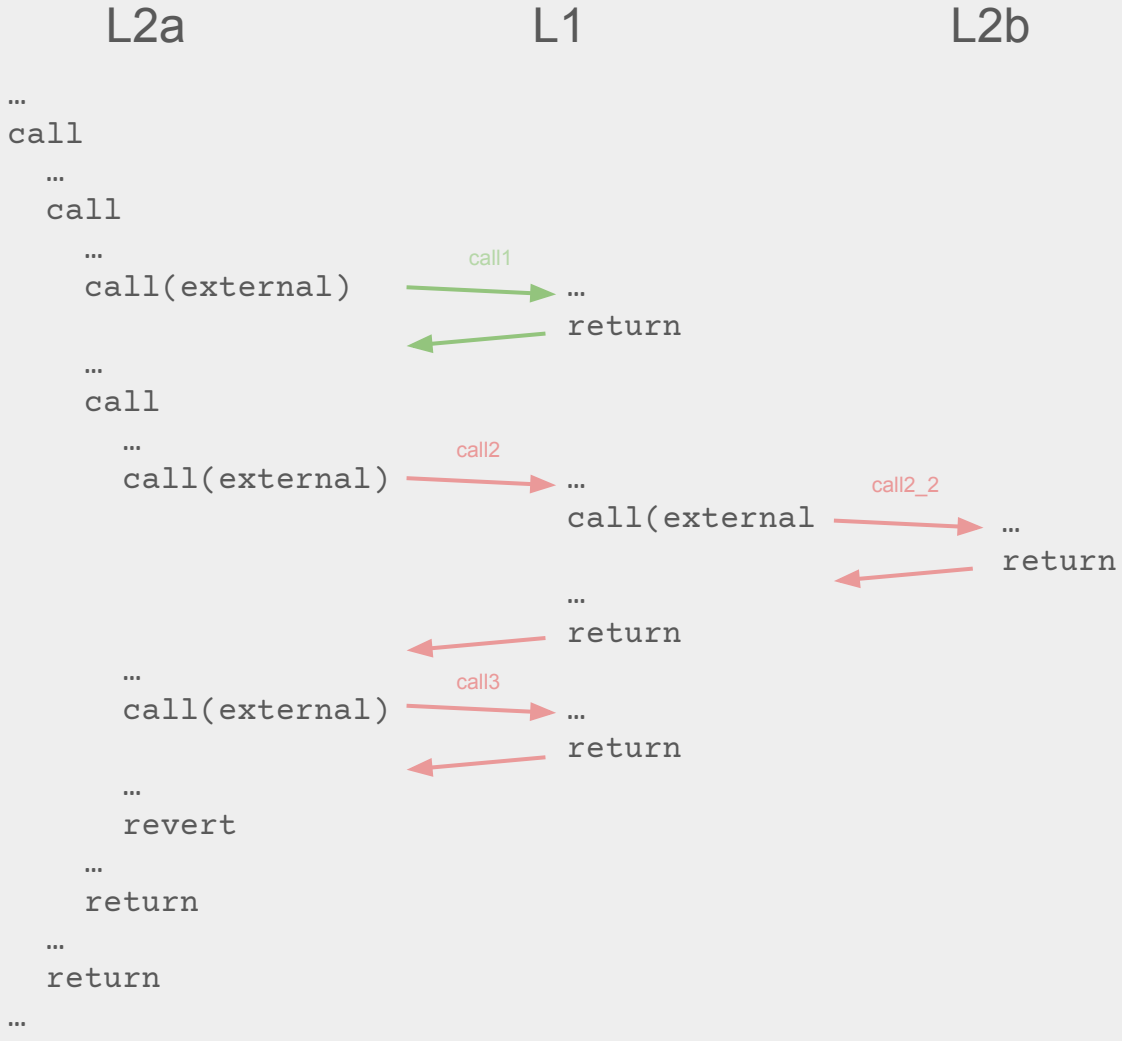
L1 call_static L2 call_static L1

```
proofSystemBatchPerVerificationEntries = {
   entries: [],
   l1ToL2lookupCalls: [{
      crossChainCallHash = keccak(
            dest rollupId: 1,
            originalAddress: 0x1234,
            value: 0,
            callData: 0xaaaaaa,
            sourceAddress: 0x5678
            src rollupId: MAINNET_ROLLUP_ID=0),
      destinationRollupId = 2,
      returnData = “myFirstStaticReturn”,
      failed = false,
      l2ToL1Calls = [{
            isStatic: true,
            targetAddress: 0x11111c,
            value: 2,
            data: 0xaaaa,
            sourceAddress: 0x22222,
            sourceRollupId: 1,
            reverSpan: 0
         }],
      expectedL1ToL2Calls = [],
      expectedLookups = [],
      callCount = 0,
      rollingHash = 0x123456
    }],
    …
}
```

L2a call L1 call/revert L2b



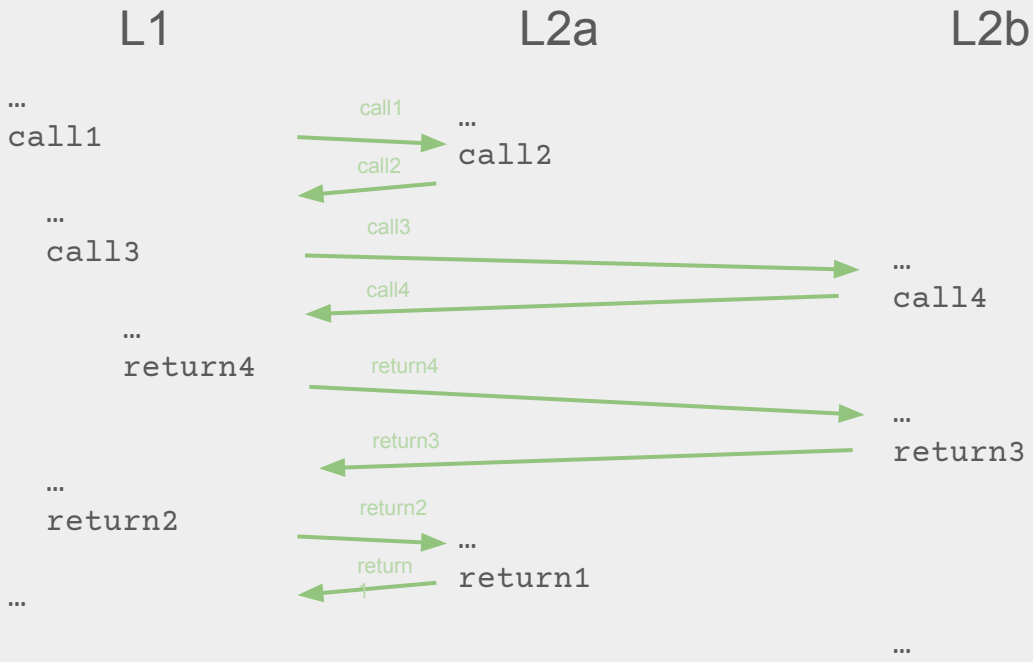

```
proofSystemBatchPerVerificationEntries = {
   entries = [{
          stateDeltas: [{1, 0x1234, 0x5678, 0}],
          proxyEntryHash: 0,
          destinationRollupId: 0,
          returnData: {}
          l2ToL1Calls: [{
              isStatic: false,
              targetAddress: 0xADDR_INL1,
              value: 2,
              data: 0xCALLDATA_CALL_1,
              sourceAddress: 0xADDR_L2a,
              sourceRollupId: 1,
              reverSpan: 0
          }],
          expectedL1ToL2Calls: [],
          expectedLookups = [{
              crossChainCallHash: keccak(
                     dest_rollupId: 1,
                     targetAddress: 0xADDR_IN_L2b,
                     value: 2,
                     callData: 0xCALLDATA_CALL1_2,
                     sourceAddress: 0xADDR_IN_L1,
                     src_rollupId: MAINNET_ROLLUP_ID=0),
              returnData: "MyRevertReturn CALL1_2"
              failed: true,
              l2ToL1CallNumber: 1
              executingLookupIndex: 0
              callCount: 0,
              rollingHash: 0
            }],
          callCount = 1,
          rollingHash = 0x646464646,
       }],
   l1ToL2lookupCalls = [],
   …
}
```


L2a
L1
L2b
…
call
…
call
…
call(external)
call1
…
return
…
call
…
call(external)
call2
…
call(external
call2_2
…
return
…
return
…
call(external)
call3
…
return
…
revert
…
return
…
return
…

```
proofSystemBatchPerVerificationEntries = {
   entries = [{
          stateDeltas: [{1, 0x1234, 0x5678, 0}],
          proxyEntryHash: 0,
          destinationRollupId: 1,
          returnData: {}
          l2ToL1Calls: [{
              isStatic: false,
              targetAddress: 0xTARGET_CALL1,
              value: 2,
              data: "data call 1",
              sourceAddress: 0xSOURCE_CALL1,
              sourceRollupId: 1,
              reverSpan: 0
          },{
              isStatic: false,
              targetAddress: 0xTARGET_CALL2,
              value: 2,
              data: "data call 2",
              sourceAddress: 0xSOURCE_CALL2,
              sourceRollupId: 1,
              reverSpan: 2
          },{
              isStatic: false,
              targetAddress: 0xTARGET_CALL3,
              value: 2,
              data: "data call 3",
              sourceAddress: 0xSOURCE_CALL3,
              sourceRollupId: 1,
              reverSpan: 0
          }],
          expectedL1ToL2Calls: [{
```

```
          expectedL1ToL2Calls: {[
              crossChainCallHash: keccak(
                  dest_rollupId: 1,
                  targetAddress: 0XTARGET_CALL_2_1,
                  value: 2,
                  callData: "DATA CALL 2 1",
                  sourceAddress: 0XSOURCE_CALL_2_1,
                  src_rollupId: MAINNET_ROLLUP_ID=0),
              returnData: "return calldata 2_3"
              callCount: 0
           }],
          expectedLookups = []
          callCount = 3,
          rollingHash = 0x646464646,
   }],
   l1ToL2lookupCalls = []
   …
}
```

L2a call L1 (call1)
L2a call L1 (call2)
L1 call L2b (call2_1)
L2a call L1 (call3)
revert


L1
L2a
L2b
…
call1
call1
…
call2
call2
…
call3
call3
…
call4
call4
…
return4
return4
…
return3
return3
…
return2
return2
…
return1
return
…
…

```
proofSystemBatchPerVerificationEntries = {
   entries = [{
         stateDeltas: [{1, 0x1234, 0x5678, 0}],
         proxyEntryHash: ,keccak(
              dest rollupId: 1, // L2a
              targetAddress: 0xCALL1_dest,
              value: 0,
              callData: 0xCALL1_data,
              sourceAddress: 0xCALL1_src,
              src_rollupId: MAINNET_ROLLUP_ID=0),
         destinationRollupId: 1,
         returnData: {}
         l2ToL1Calls: [{
             isStatic: false,
             targetAddress: 0xTARGET_CALL2,
             value: 2,
             data: "data call 2",
             sourceAddress: 0xSOURCE_CALL2,
             sourceRollupId: 1,  // L2a
             reverSpan: 0
         },{
             isStatic: false,
             targetAddress: 0xTARGET_CALL4,
             value: 0,
             data: "data call 4",
             sourceAddress: 0xSOURCE CALL4,
             sourceRollupId: 2,  // L2b
             reverSpan: 0
         }],
         expectedL1ToL2Calls: [{
```

```
         expectedL1ToL2Calls: {[
             crossChainCallHash: keccak(
                 dest_rollupId: 1,
                 targetAddress: 2, // L2b
                 value: 0,
                 callData: "DATA CALL 3",
                 sourceAddress: 0XSOURCE_CALL_3,
                 src_rollupId: MAINNET_ROLLUP_ID=0),
             returnData: "return calldata 2_3"
             callCount: 1
          }],
         expectedLookups = []
         callCount = 1,
         rollingHash = 0,
   }],
   l1ToL2lookupCalls = []
   …
}
```

L2a call L1 (call1)

L2a call L1 (call2)

L1 call L2b (call2_1)

L2a call L1 (call3)

revert

Proving system

- **Builds Sends the postAndVerifyBatch transaction/bundle. Proving systems ASSUME information from L1 via Hash**

- **The proving systems MUST check the global structure of EEZ**
  - Validate and extract information from the blobs.
  - Proving Systems MAY accept transitions from chains that don’t verify.

- **L1 blockhash and timestamp are included as part of the proving info**
  - This is very useful for querying L1 from L2 without any cross-chain calls.
  - Querying L2 state from L1 is trivial as all the L2 states are already in L1.
  - This allows bridging or ASYNC messaging very cheap.

```
function  verifyProofSystemBatch(ProofSystemBatchPerVerificationEntriescalldata batch, bytes32[][]
      memory vkMatrix) internal view {
        // Selected blob hashes (indexed into the tx-level blob set)
        bytes32[] memory blobHashes = new bytes32[](batch.blobIndices.length);
        for (uint256 i = 0; i < batch.blobIndices.length; ++i) {
            blobHashes[i] = blobhash(batch.blobIndices[i]);
        }

        // Per-entry hash binds the FULL entry content: stateDeltas, proxyEntryHash,
        // destinationRollupId, l2ToL1Calls[], expectedL1ToL2Calls[], expectedLookups[],
        // callCount, returnData, rollingHash. Prevents an orchestrator from swapping
        // call/reentrant-call/returnData at execution time without invalidating the proof.
        bytes32[] memory entryHashes = new bytes32[](batch.entries.length);
        for (uint256 i = 0; i < batch.entries.length; ++i) {
            entryHashes[i] = keccak256(abi.encode(batch.entries[i]));
        }

        // Per-lookup-call hash, same rationale.
        bytes32[] memory lookupCallHashes = new bytes32[](batch.l1ToL2lookupCalls.length);
        for (uint256 i = 0; i < batch.l1ToL2lookupCalls.length; ++i) {
            lookupCallHashes[i] = keccak256(abi.encode(batch.l1ToL2lookupCalls[i]));
        }

        bytes32 sharedPublicInput = keccak256(
            abi.encodePacked(
                abi.encode(entryHashes),
                abi.encode(lookupCallHashes),
                abi.encode(blobHashes),
                keccak256(batch.callData),
            )
        );
```

```
        // Fetch (timestamp, blockHash) for every rollup ONCE — reused across all PS loops
        // below. Both reads are `view` (STATICCALL) so a malicious manager cannot reenter.
        uint256 rollupCount = batch.rollupIdsWithProofSystems.length;
        uint256[] memory timestamps = new uint256[](rollupCount);
        bytes32[] memory blockHashes = new bytes32[](rollupCount);
        for (uint256 r = 0; r < rollupCount; r++) {
            (timestamps[r], blockHashes[r])= IRollupContract(
                    rollups[batch.rollupIdsWithProofSystems[r].rollupId].rollupContract
                ).getTimestampAndBlockHash(batch.blockNumber);
        }

        // Per-PS verification — for each PS k, walk attesting rollups in canonical order
        // (rollupId-ascending, the order the batch enforces) and fold each rollup's
        // (rollupId, vkey for PS k, blockHash, timestamp) into a rolling accumulator. Off-
        // chain provers MUST mirror this incremental scheme so the on-chain rebuild matches.
        for (uint256 k = 0; k < batch.proofSystems.length; k++) {
            bytes32 acc = bytes32(0);
            for (uint256 r = 0; r < rollupCount; r++) {
                RollupIdWithProofSystemscalldata rps = batch.rollupIdsWithProofSystems[r];
                uint256 j =  findIndexPosition(rps.proofSystemIndex, k);
                if (j == type(uint256).max) continue;
                acc = keccak256(abi.encode(acc, rps.rollupId, vkMatrix[r][j], blockHashes[r], timestamps[r]));
            }

            bytes32 publicInputsHash = keccak256(abi.encodePacked(sharedPublicInput, acc));

            if (!IProofSystem(batch.proofSystems[k]).verify(batch.proofs[k], publicInputsHash)) {
                revert InvalidProof();
            }
        }
    }
```

Proving system requirements

- All proof systems must verify:
  - Blob format and structure
  - L1 hash consistency with the blob data
  - All ADSTFs of all the chains that the PS verifies. (Assume that not verified chains are correct)

- At smart contract level, a PS just verifies a hash

Verified by all legid Proof systems

type	from	to	info
tx		L2a	rawTX
call	L2a	L1	from, to, value, data
return	L1	L2a	success, retData
call	L2a	L1	from, to, value, data
call	L1	L2b	success, retData
return	L2b	L1	success, retData
return	L1	L2a	success, retData
call	L2a	L1	success, retData
return	L1	L2a	success, retData
revert	L2a	L2a	called(-2)
end_tx	L2a	-	success

```
proofSystemBatchPerVerificationEntries = {
   entries: [],
   l1ToL2lookupCalls: [{
      crossChainCallHash = keccak(
            dest rollupId: 1,
            originalAddress: 0x1234,
            value: 0,
            callData: 0xaaaaaa,
            sourceAddress: 0x5678
            src rollupId: MAINNET_ROLLUP_ID=0),
      destinationRollupId = 2,
      returnData = "myFirstStaticReturn",
      failed = false,
      l2ToL1Calls = [{
            isStatic: true,
            targetAddress: 0x11111c,
            value: 2,
            data: 0xaaaa,
            sourceAddress: 0x22222,
            sourceRollupId: 1,
            reverSpan: 0
         }],
      expectedL1ToL2Calls = [],
      expectedLookups = [],
      callCount = 0,
      rollingHash = 0x123456
    }],
    …
}
```

Aggregating proofs with the cryptographic accumulator

- It can be seen as a Hash of a set.
- Sets can be added/subtracted easily.

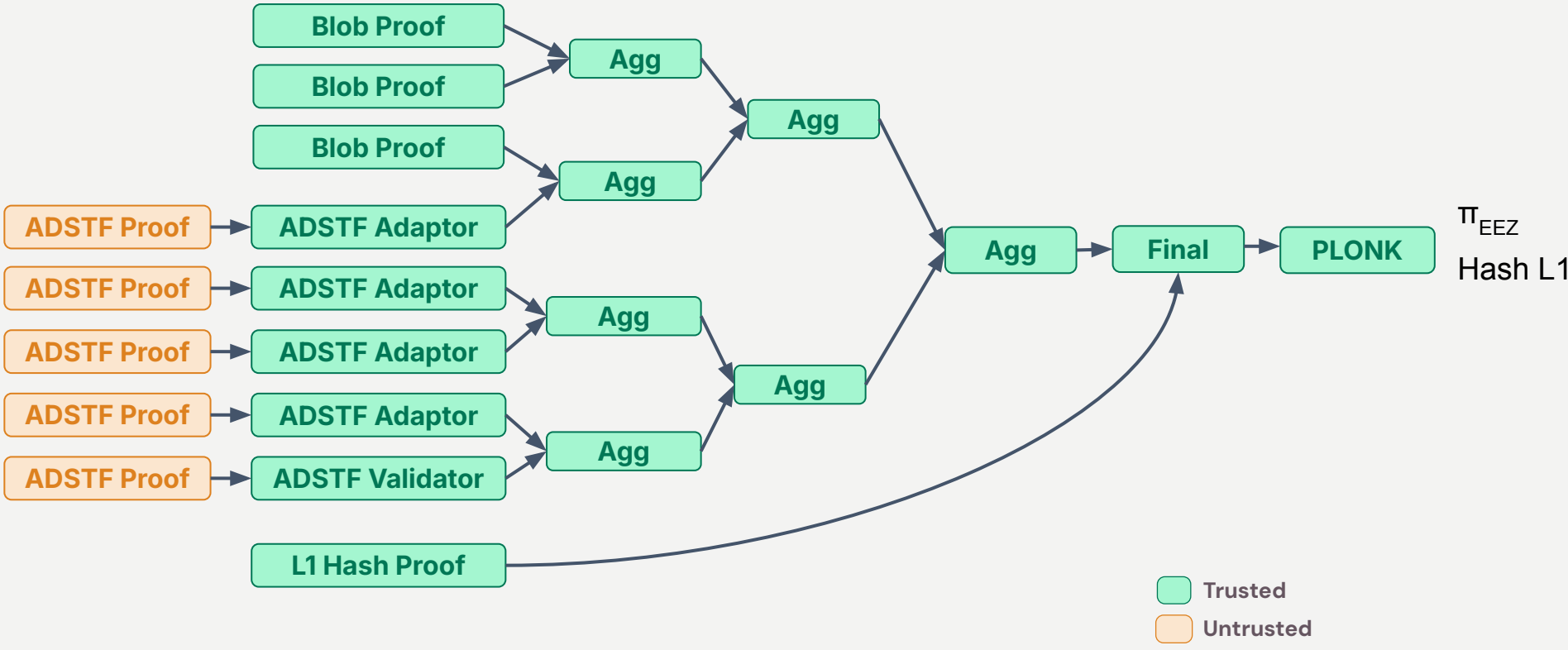
Implementation in EEZ

$$A = \sum_{i=1}^{n} s_i \cdot H(e_i) \quad \text{with } s_i \in \{+1, -1\} \text{ and } n \leq N_{\max}$$

large-hash summation with a bounded element count

THE EEZ PROOF

EEZ — ZisK proof



THE EEZ PROOF

BLOB Processor: Parses BLOB data

PROVE	-	Blob commitment
ASUME	+	Starting blob rollup state
PROVE	-	Ending blob rollup state
ASUME	+	ADSTF
ASUME	+	RollupId –> PS, VK
ASUME	+	L1 Interactions

ADSTF Adaptor: Verifies a user defined circuit VK

PROVE	**-**	**ADSTF**

THE EEZ PROOF

L1 Hash Builder: Build the L1 Hash

ASUME	+	Blob commitment
PROVE	-	Starting blob rollup state
ASUME	+	Ending blob rollup state
PROVE	-	L1 Interactions
PROVE	-	RollupId –> PS, VK

The Aggregation Circuit

(1) Verifies 2 circuits (a basic circuit or himself)

(2) Adds the accumulator of both circuits.

The Final Circuit

(1) Verifies the root aggregation

(2) Verifies the Hash L1 bulder circuit

(3) Adds the accumulator of both circuits and check the sum = 0

The PLONK Circuit

(1) Verifies the final circuit

(2) Generates a PLONK proof that’s easily verifiable onchain

The Aggregation Circuit

(1) Verifies 2 circuits (a basic circuit or himself)

(2) Adds the accumulator of both circuits.

The Final Circuit

(1) Verifies the root aggregation

(2) Verifies the Hash L1 bulder circuit

(3) Adds the accumulator of both circuits and check the sum = 0

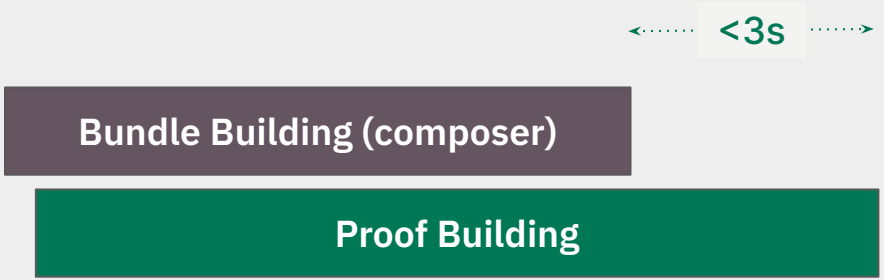
The PLONK Circuit

(1) Verifies the final circuit

(2) Generates a PLONK proof that's easily verifiable onchain

Synchronous Composability

Minimizing the proof generation latency



WHERE WE'RE HEADED

Roadmap

01 Cleanup smart contract

02 Cleanup documentation

03 Request for comments

04 Audit smart contracts

05 Signature and Zisk proving system

06 Composer 1.0

07 Chain Zero

08 Connecting Gnosis Chain

ETHEREUM
ECONOMIC
ZONE

ZisK VM

Thank you!
Q&A

DAPPCon EEZ Workshop, 17 June 2026

@jbaylina